



## 1 Contexte général

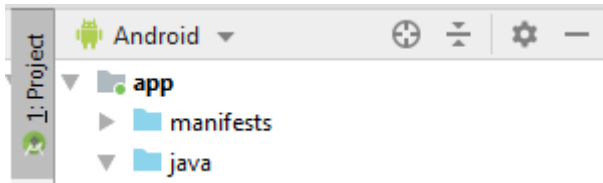
Nous allons créer étape par étape une application nommée **LocDVD** de sélection de films classés par catégorie.

Nous allons créer une **interface** avec des **boutons**, des **menus** et des **sous menus**.

## 2 L'interface de démarrage

La page de démarrage propose un choix de types de films.

Mode d'affichage (en haut à gauche) : Project : complet / Android : simplifié



Nous allons donc créer l'interface suivante :





## 2.1 Définition du layout principal (écran principal)

Un **layout** est un écran qui peut être :

- un **LinearLayout** : les éléments sont alignés horizontalement ou verticalement.
- ou un **ConstraintLayout** : les éléments sont placés en fonction des autres éléments.

A la création d'un projet, un **layout** de type **constraint** avec un **TextView** est automatiquement créé pour afficher un « Hello world ». Il se situe dans : **res/layout**. Il se nomme : **activity\_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World !"
        android:textSize="30sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

Pour plus d'informations :

<https://developer.android.com/reference/androidx/constraintlayout/widget/ConstraintLayout.html?hl=fr>

<https://developer.android.com/reference/android/widget/LinearLayout.html>



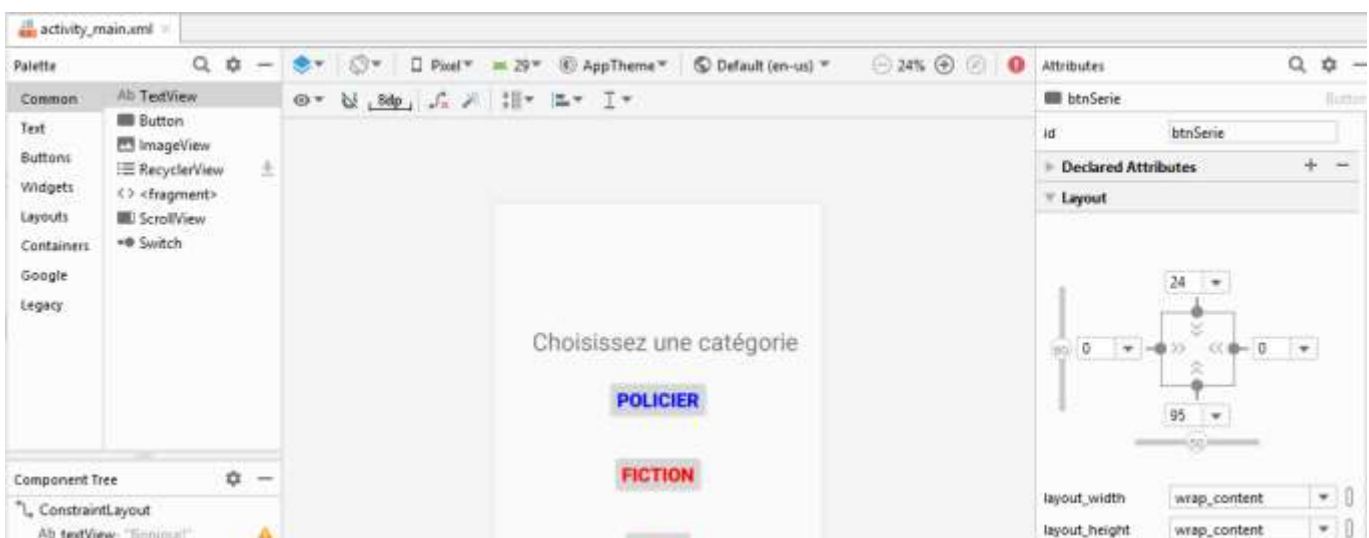
## 2.2 Gestion des boutons

### 2.2.1 Création et position des boutons

- Conserver le **TextView** que vous déplacerez vers le haut en le modifiant en « Choisissez une catégorie » taille de police : 34sp
- Ajouter 4 widgets **Button** dans l'éditeur XML en mode graphique. Il suffit de cliquer et de tirer pour créer des boutons



- Accéder aux **Attributes** de l'interface





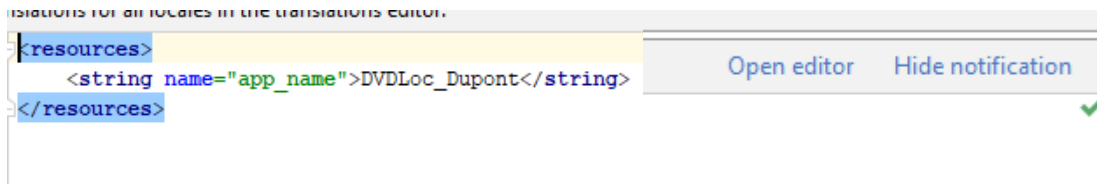
- Définir la propriété **id** de chaque bouton et nommez les ainsi : **btnPolicier**, **btnFiction**, **btnDocu**, **btnSerie**
- Redimensionner chaque bouton et définir les contraintes de positionnement :
  - Marges gauche et droite : 21dp
  - Espacement entre boutons 24dp
  - Espacement entre 1<sup>er</sup> bouton et texte : 64dp
- Changer la taille de la police des boutons à 434sp

Le **code XML** est généré automatiquement pour chaque bouton (onglet au bas de la fenêtre).

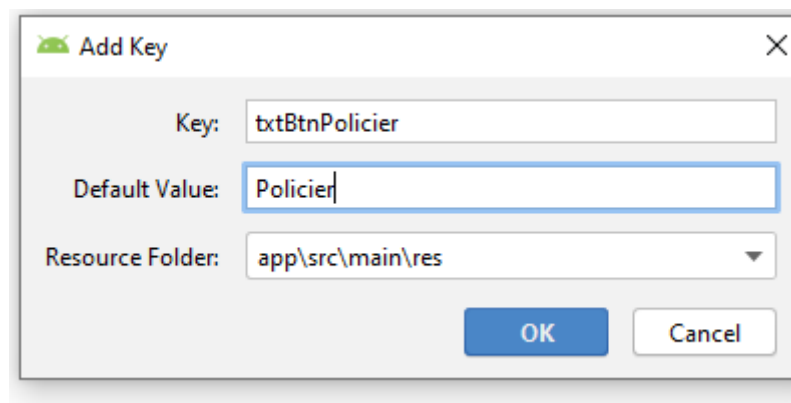
### 2.2.2 Définition des valeurs de texte des boutons

Les textes des boutons ne doivent pas être saisis « en dur » mais enregistrés en **strings**.

- Ouvrir le répertoire **res/values**
- Ouvrir le fichier **strings.xml** puis sur le lien **Open editor**



- Pour chaque bouton, cliquer sur +, puis sur **String**, ensuite définir la **Key** et la **Default Value**.



- Les chaînes de caractères sont automatiquement ajoutées dans le fichier **Strings.xml** associé.

```
<resources>
    <string name="app_name">DVDLoc_Dupont</string>
    <string name="txtBtnPolicier">Policier</string>
</resources>
```



### 2.2.3 Gérer le texte des boutons

Revenir dans **res/layout/activity\_main.xml**

- Affecter à la propriété **android:text** de chaque bouton le nom de la ressource correspondante, soit **txtBtnPolicier**,....

```
<Button
    android:id="@+id/btnPolicier"
    android:layout_width="350sp"
    android:layout_height="100sp"
    android:layout_marginStart="21dp"
    android:layout_marginTop="64dp"
    android:layout_marginEnd="21dp"
    android:text="@string/txtPolicier"
    android:textSize="34sp"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.526"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/txtMenu" />
```

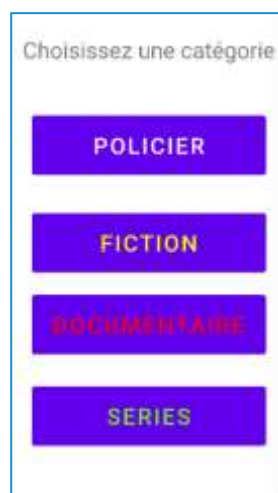
### 2.2.4 Définition des couleurs des boutons

- Ouvrir le répertoire **res/values**
- Ouvrir le fichier xml : **color**
- Ajouter 3 entrées en définissant les couleurs rouge, vert et jaune. Exemple : Rouge #FF0000.

Le code XML du fichier de ressources **color.xml** devra ressembler à ce qui suit :

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="rouge">#FF0000</color>
    <color name="vert">#74FA05</color>
    <color name="jaune">#FFFF00</color>
</resources>
```

Utiliser ensuite, dans **activity\_main.xml**, la propriété **TextColor** pour choisir la couleur





## 2.3 Modification du thème

Modifier le thème de l'application dans le fichier AndroidManifest.xml pour faire apparaître l'action bar.

```
android:label="My Application"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Base.Theme.AppCompat.Light.DarkActionBar"
tools:targetApi="11">
<activity
    android:name=".MainActivity"
```

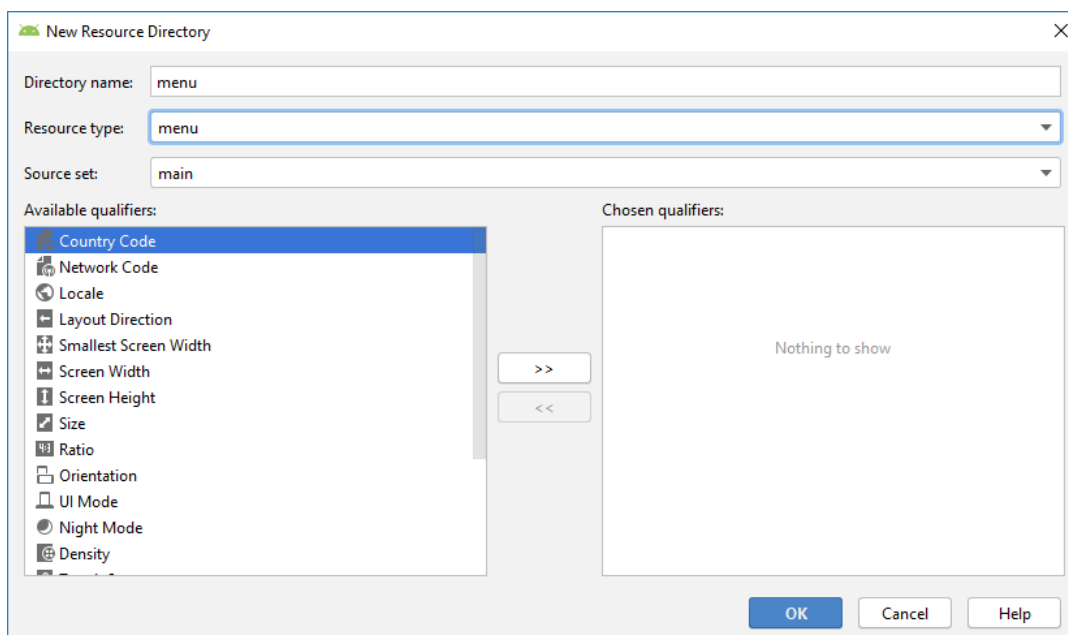
## 2.4 Création de menus

Nous allons créer 4 entrées de menus :

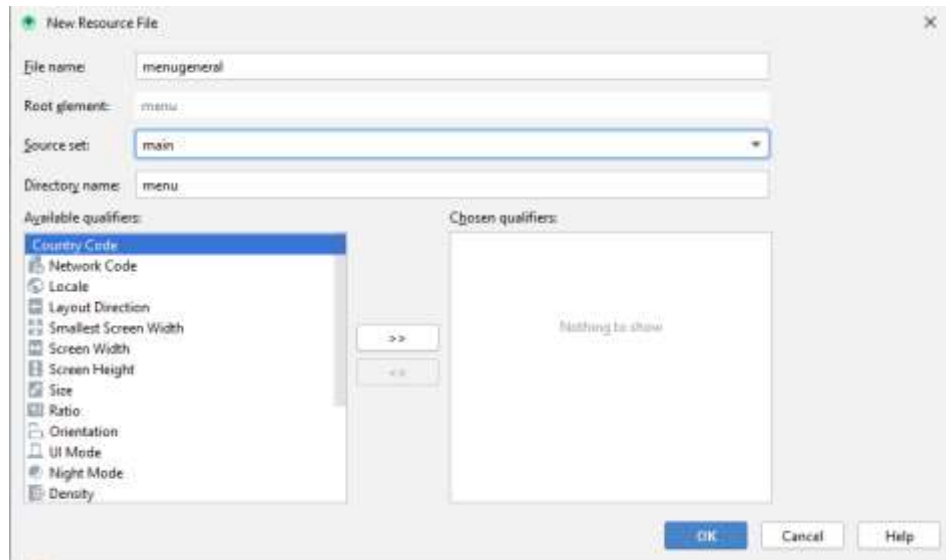
- Rechercher un film
- Réserver un film
- Magasins (liste des)
- Présentation (de la société)

Pour cela :

- Cliquer droit sur le dossier **res/menu** dans l'arborescence
- Si le dossier n'existe pas, faire **New / Android Resource Directory** (clic droit sur le dossier res puis new).



- Créer un **nouveau fichier Menu « Menu Resource file »** (clic droit sur le dossier **menu** puis **new**)
- Le nommer : **menugeneral**



- Cliquer sur **OK**
- Ajouter un **menu item** depuis la palette et lui donner l'identifiant **menuRechercher**
- Créer une entrée supplémentaire dans le fichier strings.xml :  
`<string name="txtMenuRecherche">Rechercher un film</string>`
- Définir le **titre** du menu : `@string/txtMenuRecherche`



- Procéder de même pour les autres items de menu : **Réserver un film**, **Magasins**, **Présentation**.

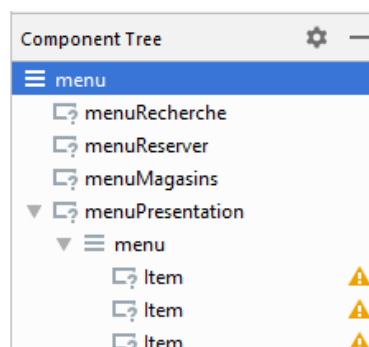
## 2.5 Création de sous menus

Dans le menu **Présentation de la société**, nous ajoutons 3 sous menus :

- Contact
- L'équipe
- Croissance

Pour créer les sous menus :

- Ajouter un élément **menu** depuis la palette sur l'item **Présentation**
- Ajouter ensuite trois **Menu item** sous l'entrée **menu**





- Procéder comme ci-dessous pour les titres et les identifiants de chaque entrée

Ce qui génère dans le fichier **menu.xml**, le code suivant :

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:app="http://schemas.android.com/apk/res-auto"
      xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/menuRechercher"
        android:title="Rechercher un film" />
    <item
        android:id="@+id/menuReserver"
        android:title="Reserver un film" />
    <item
        android:id="@+id/menuMagasins"
        android:title="Magasins" />
    <item
        android:id="@+id/menuPresentation"
        android:title="Presentation" >
        <menu >
            <item
                android:id="@+id/menuContact"
                android:title="Contact" />
            <item
                android:id="@+id/menuEquipe"
                android:title="L'équipe" />
            <item
                android:id="@+id/menuCroissance"
                android:title="Croissance" />
        </menu>
    </item>
</menu>
```

## 2.6 Activation des menus et sous menus

### Activation des menus

Pour pouvoir activer le menu lors de l'appui sur la **touche Menu** du téléphone ou de l'émulateur :

- Ouvrir dans le dossier **java** le fichier **MainActivity.java**
- Créer ou modifier la méthode Java **onCreateOptionsMenu** pour activer le menu. Copier le code ci-dessous à la place du code généré.
- Ajouter les « **import** » nécessaires **android.view.Menu** et **android.view.MenuInflater**.  
Faire **Alt / Entrée** sur le nom de l'élément en erreur nécessitant un import





```
}//fin onCreate
```

```
//Menu
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.menugeneral, menu);
    return true;
}
```

La classe **MenuInflater** gère l'ouverture et la fermeture du mode menu de l'application par le téléphone. A ce stade le menu devrait s'afficher en haut de la fenêtre :



### Activation des items du menu

Quand l'utilisateur choisit un item dans le menu, le système appelle la méthode **onOptionsItemSelected()** qui reçoit le **MenuItem** sélectionné.

Implémenter la méthode **onOptionsItemSelected** dans le fichier **src/MainActivity.java**.

Penser à enregistrer tous les fichiers xml que vous avez créé ou manipulé.

```
// Activation des menus et sous menus
public boolean onOptionsItemSelected(MenuItem item)
{
    switch (item.getItemId())
    {
        case R.id.menuRechercher:
            Log.i( tag: "LocDVD", msg: "Menu:Rechercher un film");
            return true;

        case R.id.menuReserver:
            Log.i( tag: "LocDVD", msg: "Menu:Réserver un film");
            return true;

        case R.id.menuMagasins:
            Log.i( tag: "LocDVD", msg: "Menu:Magasin");
            return true;

        case R.id.menuContact:
            Log.i( tag: "LocDVD", msg: "Menu:Contact");
            return true;
    }
    return true;
}
```



### Informations :

La classe **android.util.Log** inclut des méthodes d'aide au développement (comme printf) :

- **Log.v** : Affiche le message en mode "*verbose*" c'est à dire "*verbeux*" ou "*abondant*" en français
- **Log.d** : Affiche le message en mode "*debug*", utilisé pour le débogage
- **Log.e** : Affiche le message en mode "*error*" (erreur)
- **Log.w** : Affiche le message en mode "*warning*", c'est à dire les avertissements
- **Log.i** : Affiche le message en mode "*info*"

Le menu cliqué s'affiche dans le **LogCat** du **DDMS**.

Exemple : 2022-02-08 09:29:52.014 2378-2378/com.example.dvd I/LocDVD: Menu:Réserver un film

NB : Si l'onglet **LogCat** ne s'affiche pas au bas de l'éditeur, faire : Windows / Show View/ Other / Android / LogCat

Si erreur : « **constant expression required R.id** », ajouter **android.nonFinalResIds=false** dans le fichier gradle.properties et faire **File / Sync project with gradle**



## Affichage d'un Message à l'écran

Lorsque l'utilisateur sélectionne une option du menu, il est possible grâce à la classe Toast (bulle) d'afficher un message à l'écran.

Ici le message correspondra au titre de l'option choisie.

- Supprimer ou commenter les log.i
- Ajouter avant le switch une instruction de création d'un **Toast** :

```
// Activation des menus et sous menus
public boolean onOptionsItemSelected(MenuItem item)
{
    Toast.makeText(this, item.getTitle(), Toast.LENGTH_SHORT).show();

    switch (item.getItemId())
```



## INFORMATION : En cas de non-affichage du menu :

S'il est placé juste au-dessus du <Text View>, une erreur liée à la contrainte est affichée pour nous indiquer qu'il n'y a pas de contrainte verticale sur l'élément. Pour éviter cette "erreur" sans rajouter de contrainte manuellement, le placer au-dessus du <androidx.constraintlayout.widget.ConstraintLayout>

```
<com.google.android.material.appbar.AppBarLayout
android:id="@+id/appbar"
```

```
android:layout_width="match_parent"
android:layout_height="43dp">
```

```
<com.google.android.material.appbar.MaterialToolbar
android:layout_width="match_parent"
android:layout_height="match_parent"
app:title="Test"
app:menu="@menu/menugeneral"/>
</com.google.android.material.appbar.AppBarLayout>
```



### 3 Annexe : Quelques propriétés des layouts et widgets

**Orientation** : alignement des éléments contenus dans ce **Layout**

- **android:orientation="vertical"** : oriente les éléments sur une ligne verticale.
- **android:orientation="horizontal"** : oriente les éléments sur une ligne horizontale.

**android:width** et **android:height** : occupation de chaque bouton et du **TextView** dans le conteneur

- **wrap\_content** : la taille du bouton est ajustée à la taille du texte contenu.
- **match\_parent** : le bouton occupe toute la largeur du conteneur parent

**android:gravity** : alignement d'un élément

- Dans un **LinearLayout**, les éléments sont alignés de gauche à droite et de haut en bas. **gravity** permet de placer un élément tout en bas ou tout à droite.
- Les gravités les plus couramment utilisées sont **left**, **center\_horizontal**, **top**, **bottom**, **right**. On peut aussi utiliser **center\_vertical** pour centrer verticalement l'élément.

## Table des matières

|       |                                                           |    |
|-------|-----------------------------------------------------------|----|
| 1     | Contexte général.....                                     | 1  |
| 2     | L'interface de démarrage .....                            | 1  |
| 2.1   | Définition du layout principal (écran principal) .....    | 2  |
| 2.2   | Gestion des boutons .....                                 | 3  |
| 2.2.1 | Création et position des boutons .....                    | 3  |
| 2.2.2 | Définition des valeurs de texte des boutons .....         | 4  |
| 2.2.3 | Gérer le texte des boutons .....                          | 5  |
| 2.2.4 | Définition des couleurs des boutons .....                 | 5  |
| 2.3   | Création de menus.....                                    | 6  |
| 2.4   | Création de sous menus.....                               | 7  |
| 2.5   | Activation des menus et sous menus .....                  | 8  |
| 3     | Annexe : Quelques propriétés des layouts et widgets ..... | 12 |
|       | Table des matières.....                                   | 12 |